ICT ACADEMY OF KERALA

Summer Internship Report

LabFlow

Laboratory Equipment Management

Milan KB
Register No: SHR25CS176 | Sr No: 225763

Sahrdaya College of Engineering and Technology, Kodakara

Prepared 25 October 2026

EXECUTIVE SUMMARY

LabFlow is a full-stack MERN application for managing individually tagged equipment in a college laboratory. It brings discovery, borrowing decisions, physical handover, inspected return and damage history into a shared system. Students can access their own requests; administrators manage inventory and circulation. The implementation uses React/Vite, Express and MongoDB/Mongoose with server validation, role checks and database transactions.

The local application completes the core borrowing journey and passes 53 automated tests. Real MongoDB tests exercise simultaneous approvals, conflicting actions and rollback after failed writes. Earlier local accessibility scans found no violations or incomplete findings; keyboard and responsive samples were also checked. The application is also published on Vercel Hobby with Atlas Free. All 32 isolated Atlas acceptance cases pass on 24 October 2026, and the public API/security checks and student/admin browser journey were verified on 23 October 2026. College approval and submission remain pending; remaining browser checks are identified below.

The equipment/purpose dropdowns and unlisted assignment flow have real database coverage. The instrument-register redesign and its student request/admin assignment/issue/return journey passed local browser verification on 25 October. The new form has no axe violations at 320px; the patterned background leaves a manual contrast review item described in QUALITY. The redesign/dropdowns are published on Vercel from source 684af3c. Current public read-only checks, student login and session reload pass; a fresh live borrowing write test was deliberately skipped. Earlier hosted lifecycle evidence below describes the preceding release. The locked parser security update restores a clean dependency audit.

INTRODUCTION

Laboratory lending involves both digital decisions and physical equipment movement. A pending request should not make an asset unavailable, while an approved reservation must prevent another allocation. Informal records can obscure this distinction and lose the relationship between a borrower, return inspection and current condition.

LabFlow models these transitions explicitly. Equipment is tracked per physical asset instead of as a bulk quantity. A request preserves its original asset snapshot and chronological events so archival does not erase history. The project is an individual ICTAK capstone, providing a clean architecture for laboratory lending operations.

OBJECTIVES

1. Maintain an accurate searchable inventory and asset state.

2. Give students a clear request, cancellation and personal-history flow.
3. Give administrators controlled decisions, issue, inspection and maintenance actions.
4. Prevent cross-user access, unauthorized role changes and conflicting allocations.
5. Implement a responsive MERN application with reproducible verification and clear documentation.

SCOPE AND DELIVERABLES

The application supports one laboratory and two roles. Students register with name/email/password, sign in, browse non-archived assets, select one available asset or describe an unlisted need, select a purpose or specify Other, cancel eligible requests and report damage while borrowing. Administrators create/edit/maintain/archive assets, assign a registered available asset before approving an unlisted need, approve or reject, record issue/return, resolve reports and view current counts. Loans are limited to 168 hours from approval. Times are stored in UTC and displayed in Asia/Kolkata.

Deliverables available locally include the application source, npm workspace/lockfile, SRS document, API/security/design/quality documents, regression tests, report screenshots, CI definition and a verified public deployment with its deployment runbook. Multi-lab operation, payments, notifications, uploads, QR scanning, renewals and advanced analytics are excluded.

Student story: As a student, I want to describe an unlisted instrument so that staff can identify a suitable asset. Acceptance requires a 10–500 character description, purpose, future requested return time and stable request key. Submission creates no asset or reservation; the owner may cancel before handover.

Administrator story: As an administrator, I want to assign an available registered asset and approve the request so that the instrument has one consistent reservation. Assignment, snapshot, approval, reservation and event must commit together. Competing approval or a failed write must leave no partial allocation. The SRS defines the remaining feature acceptance criteria.

METHODOLOGY

Development followed documented requirements and small verified increments in Git, preserving the initial M0 history. The specification defines the accepted scope, business rules, selected implementation policies and verification criteria.

The browser calls a relative JSON API; only the server accesses MongoDB. Strict boundary schemas reject unknown fields and invalid IDs/dates. Authorization uses the stored session role and owner-scoped database queries. MongoDB transactions and unique partial indexes protect allocation; optimistic

equipment versions detect stale edits. A real replica set, rather than browser mocks, verifies database behavior.

Passwords use salted asynchronous scrypt. MongoDB-backed sessions regenerate at login and expire absolutely after eight hours. Exact-origin/custom-header CSRF checks and an atomic ten-attempt fixed-window authentication limit protect mutations. Deployment secrets and local development passwords remain outside Git.

PROJECT ACTIVITIES

The supplied ICTAK project guidelines and report template informed the requirements and report structure. Both original files remain unchanged.

The foundation established validated configuration, readiness, the React shell, a persistent local replica set and reproducible commands. Identity added registration, session behavior and controlled administrator provisioning. Inventory and circulation then added guarded state transitions and the corresponding student/admin screens.

Verification found and corrected several concrete issues. Mongoose transaction retries failed on an undefined embedded version field; explicit unversioned subdocument schemas retained parent concurrency control. Live deployment revealed Vercel forwarding a named rewrite capture as an extra query field; an unnamed wildcard fixed routing while preserving strict input rejection. A fresh clone exposed a concurrent first-download lock failure; explicit MongoDB preparation now precedes parallel tests. Production serving from a hidden checkout exposed an absolute-file dotfile guard; a fixed serving root resolved it and a regression fixture covers that location.

Accessibility testing found a mismatch between the logo's visible and accessible text. A decorative vector mark resolved the mismatch without disabling checks. The skip link was also fixed to focus content without changing the application route.

The final visual update required explicit Google Fonts source allowances in the production content security policy, a navigation landmark for pagination and darker accent/placeholder text. The release audit then identified GHSA-27r3-jxmh-p828; updating the one locked body-parser package to 2.4.0 restored a zero-finding audit without enabling dependency scripts.

RESULTS & FINDINGS

Verification	Observed result and scope	Evidence
Regression	16 fast server, five client and 32 real database integration tests pass; lint/build pass	npm run check
Remote CI	Checks, coverage gates and audit passed for deployed source 684af3c	GitHub Actions (https://github.com/milan-kb/LabFlow/actions/runs/40130494194)
Critical coverage	99.05% lines and 92.58% branches; existing 80% gates enforced	npm run test:coverage; quality baseline
Concurrency and rollback	One approval winner; consistent cancellation/report-return races; injected approval/return failures roll back	Domain integration suite
Actual browser journey	Redesigned unlisted request/assignment/issue/return passed locally; preceding hosted lifecycle acceptance passed on 23 October; current live write test skipped	evidence/hosted-acceptance.json; current screenshots
Automated accessibility	Current borrowing form: zero axe violations and no horizontal overflow at 320px; contrast incomplete results retained for manual review	Local axe helper; QUALITY.md
Keyboard and reflow	Core keyboard flow, 320px samples and 200% CSS content enlargement checked	Historical verification at f6af63a
Fresh setup	A fully cold clone at 94bc7e9 installed, downloaded the pinned binary, and passed check/coverage	Historical verification at f6af63a
Performance	30 warm sequential reads per endpoint: equipment p95 6.12ms, requests 7.03ms, dashboard 6.27ms; login sample 290.98ms	evidence/performance.json
Production process	Built assets/deep links and secure sessions checked locally after pruning development dependencies	evidence/production-smoke.json

The measurements use an isolated test loopback dataset and cannot predict load behavior or a sleeping free host. Native browser zoom, standalone stable Chromium and other-engine checks are not established by CSS enlargement or in-app testing alone. Screen-reader testing is also not claimed. All 32 isolated Atlas acceptance tests passed on 24 October 2026; cleanup left their three databases empty. Public HTTPS, secure cookies, strict parsing/CSRF, role/ownership checks, cancellation, rejection and both return paths passed on Vercel Hobby. The live browser journey covers request, approve, issue, report damage, damaged return and restoration. One readiness request returned 503 and recovered through the UI retry; ten subsequent sequential and five concurrent health requests returned 200. Its root cause was not established, and no availability or load-test claim is made. Render was abandoned when it required a card; no paid service was created. The owner authorized the deployment network configuration on 23 October 2026; TLS, credentials and database roles remain enforced.

The appendix illustrates the current student and administrator screens using the application environment. Original-resolution captures are also supplied separately with the submission package.

CONCLUSION

The local implementation demonstrates the intended single-lab borrowing workflow and verifies its main authorization and transaction invariants. Testing against real concurrent database operations and a cold setup exposed failures that a simple mockup would have missed. The regression suite keeps these invariants reproducible as the application changes.

The deployed application and source repository provide a reproducible reference implementation of the application system. Future improvements include broader browser and screen-reader verification, operational monitoring and a defined retention policy before any real-student deployment.

APPENDIX

Requirements: SRS_DRAFT.md revision 0.7 and its matching submission copy.

Design and contracts: DESIGN.md, API_DETAILS.md, SECURITY.md, QUALITY.md.

Evidence: evidence/.

Reproduction: root README, package lockfile and documented npm commands.

Live release and operating instructions: DEPLOYMENT.md, root vercel.json.

Business rules and implementation choices: DECISIONS.md.

Source references: the unchanged local ICTAK project guidelines and internship report template. The supplied report reference defines the nine sections used here; its sample names/date are not project facts.

Student walkthrough: sign in or register, open Equipment, choose Request equipment, select an available asset or Other, select a purpose or provide Other text, and choose a future return time. Follow the request in My requests. An eligible request can be withdrawn before handover; an issued item supports a damage report.

Administrator walkthrough: sign in using privately shared application credentials, review pending requests, and assign an available registered asset for an unlisted need. Register a missing asset first if necessary. Approve with a due time within 168 hours, record physical handover, then inspect and return the asset. Damage or an existing report requires an inspection note. The application administrator has no Atlas/cloud administrator privileges.

Reviewer credentials must be shared privately and excluded from submission archives. No reusable password appears in this report or its screenshots. The README lists environment variable names and startup commands; actual credentials stay outside Git.

Figure 1. Sign-in screen and laboratory access information. LabFlow application interface, 25 October 2026.

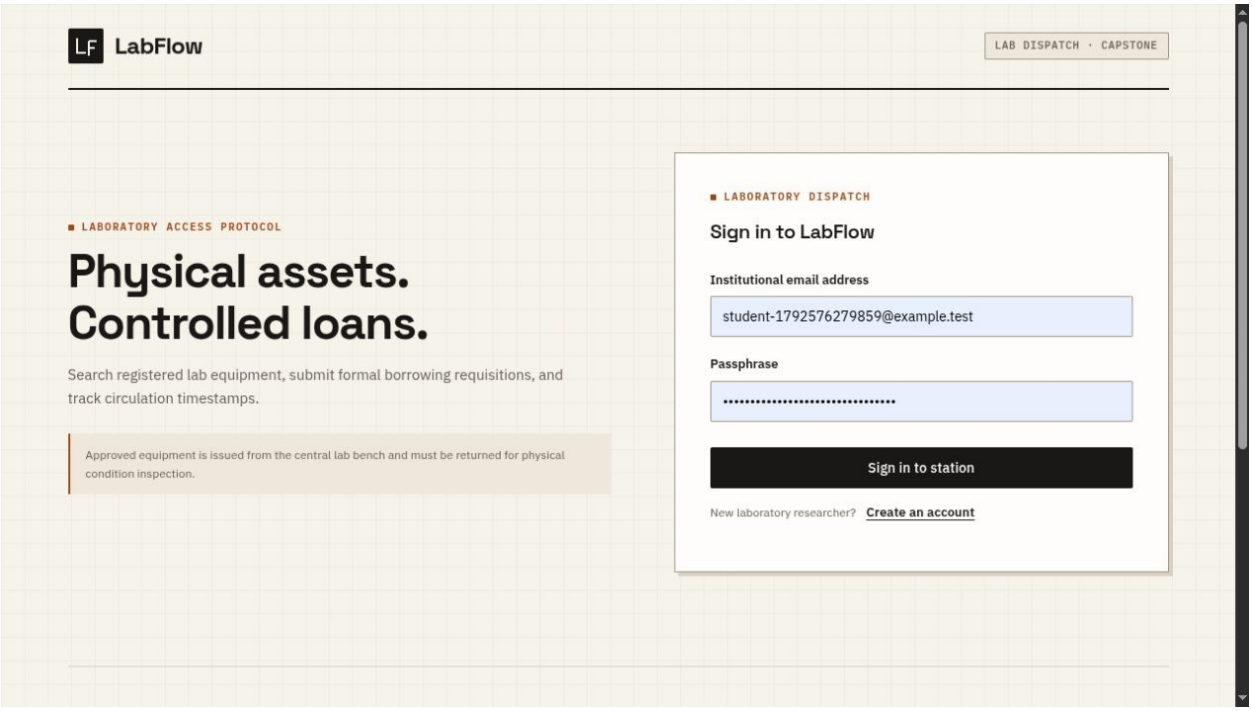


Figure 2. Student equipment catalog with availability and search filters. LabFlow application interface, 25 October 2026.

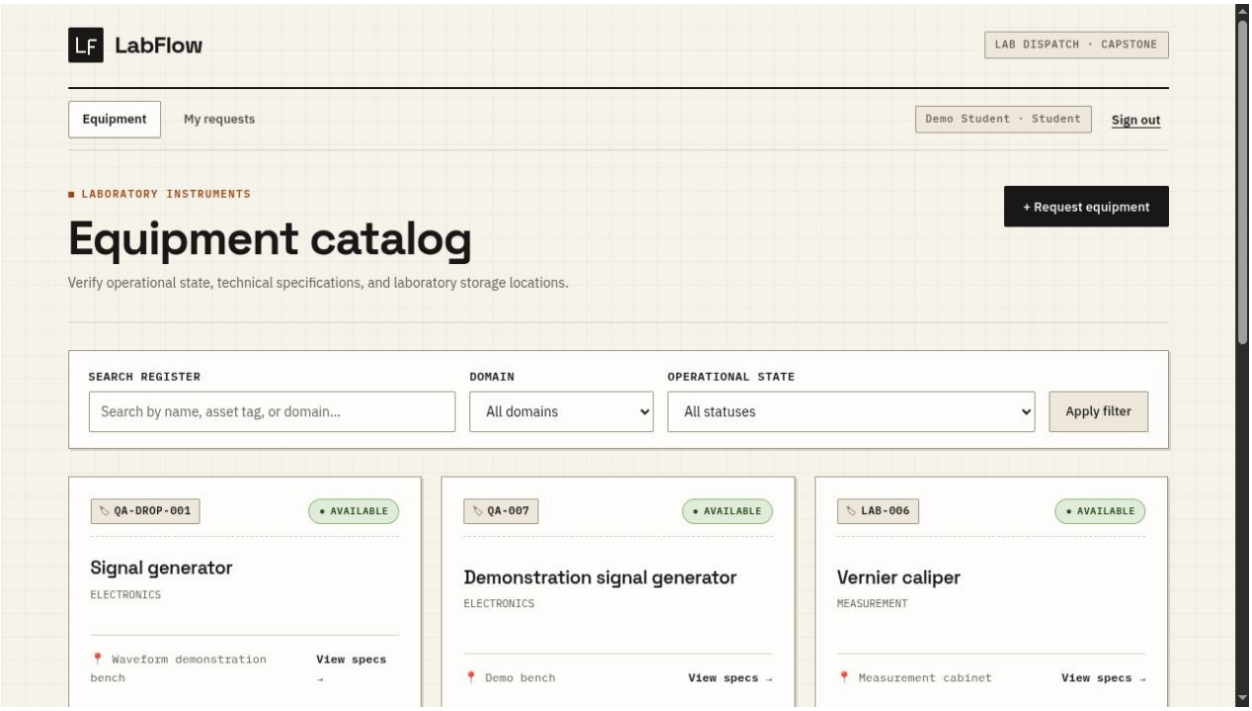


Figure 3. Equipment and purpose dropdowns with Other descriptions. LabFlow application interface, 25 October 2026.

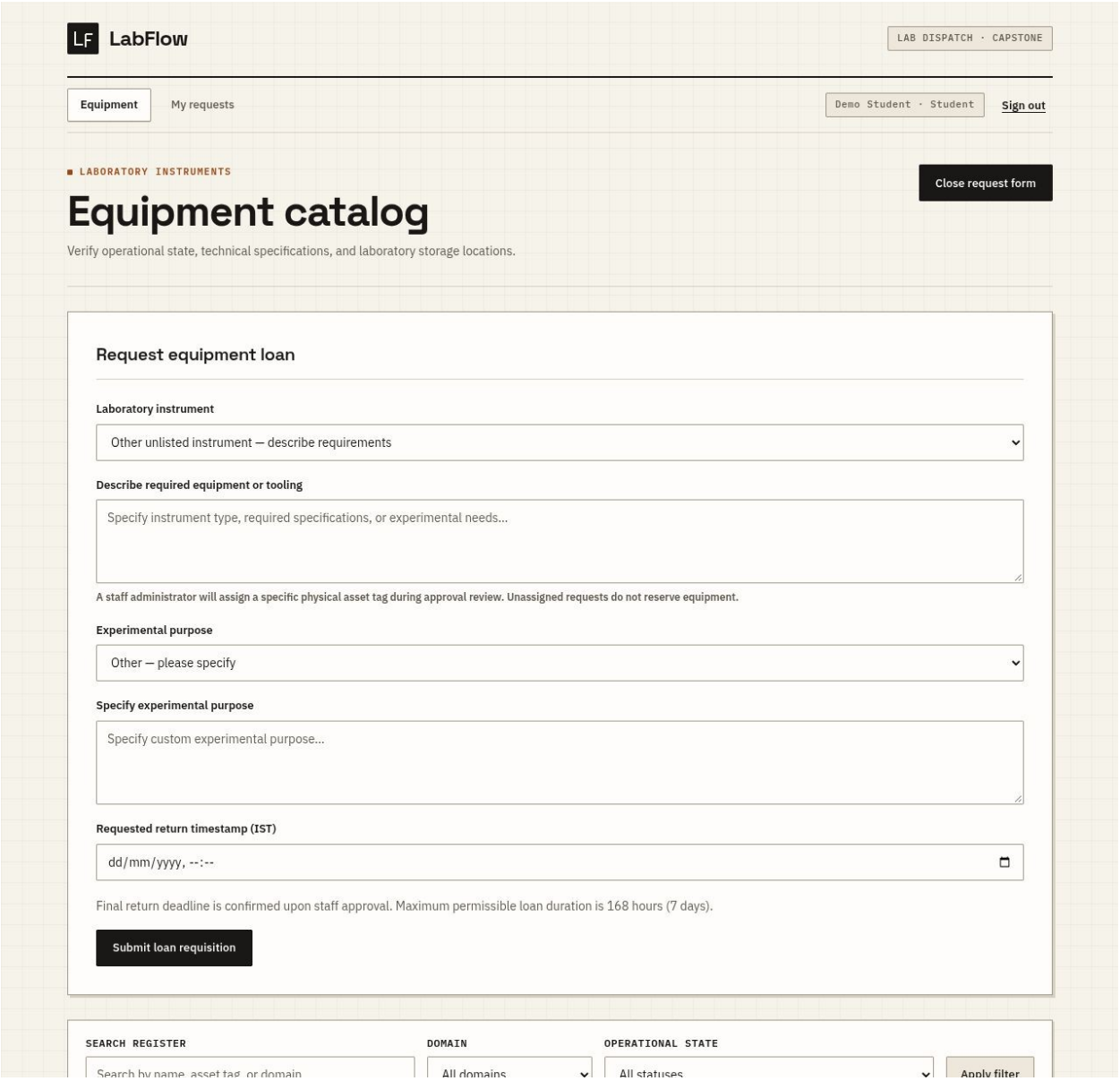


Figure 4. Student borrowing history and request states. LabFlow application interface, 25 October 2026.

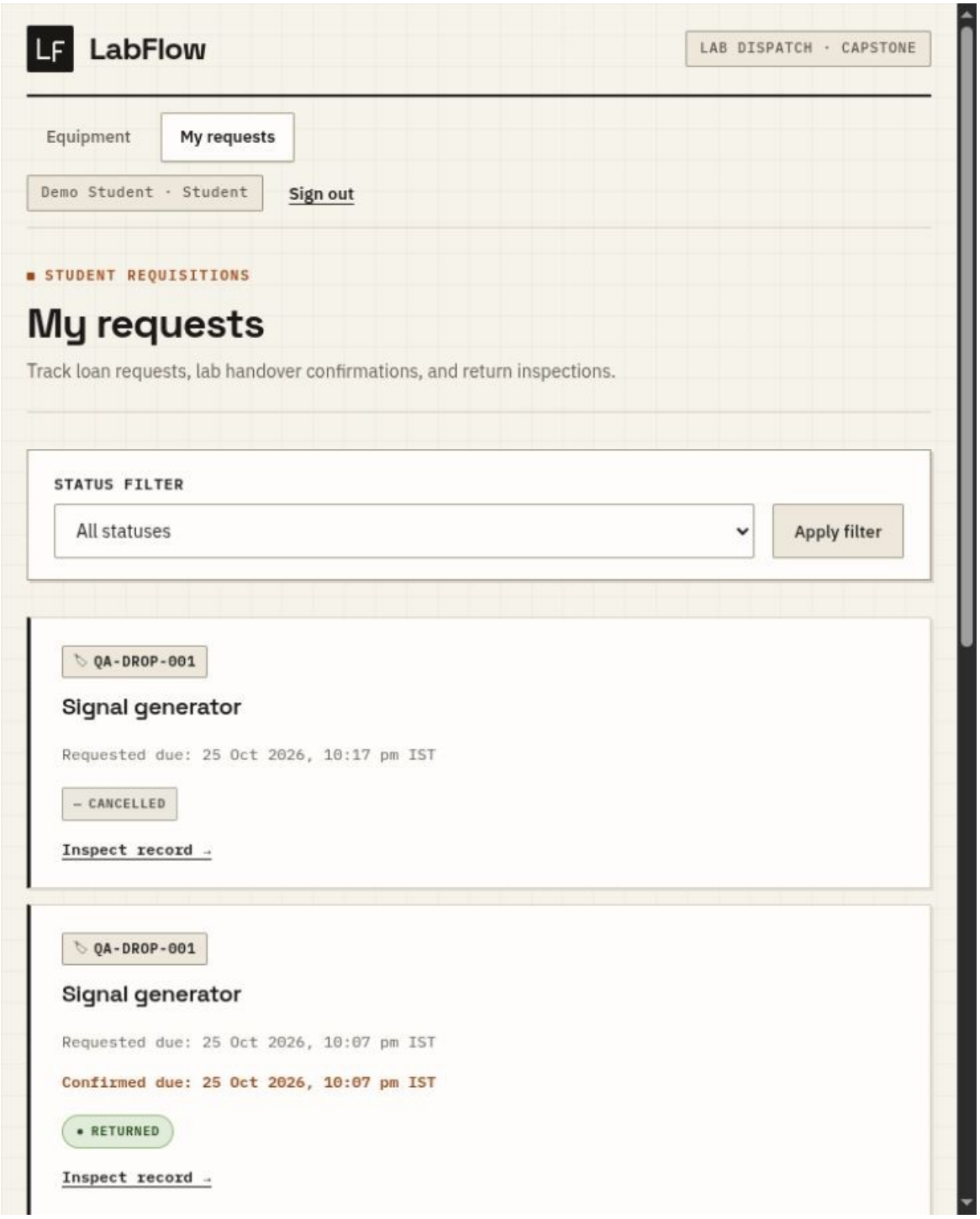


Figure 5. Administrator circulation summary and inventory state counts. LabFlow application interface, 25 October 2026.

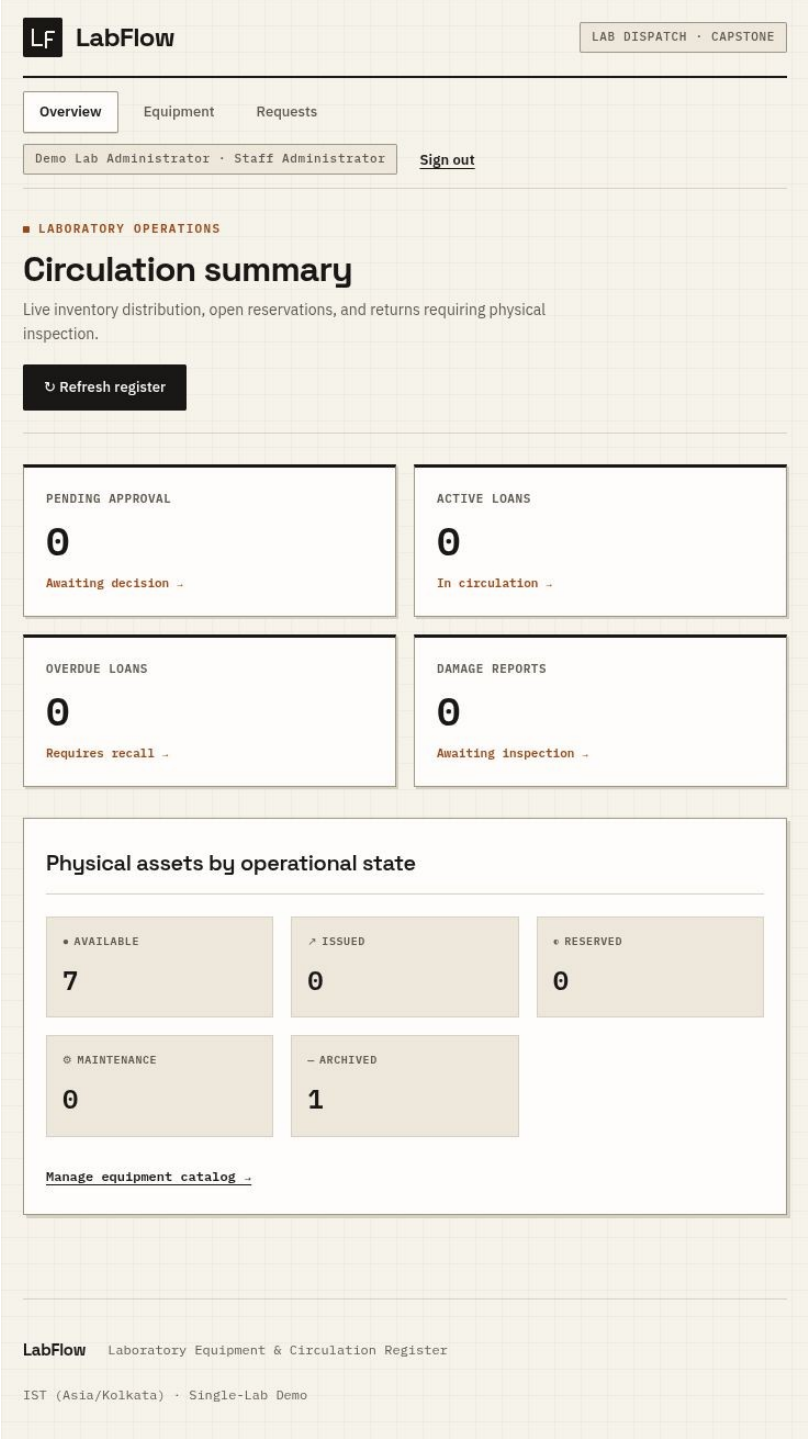


Figure 6. Administrator asset registration with preset metadata choices. LabFlow application interface, 25 October 2026.

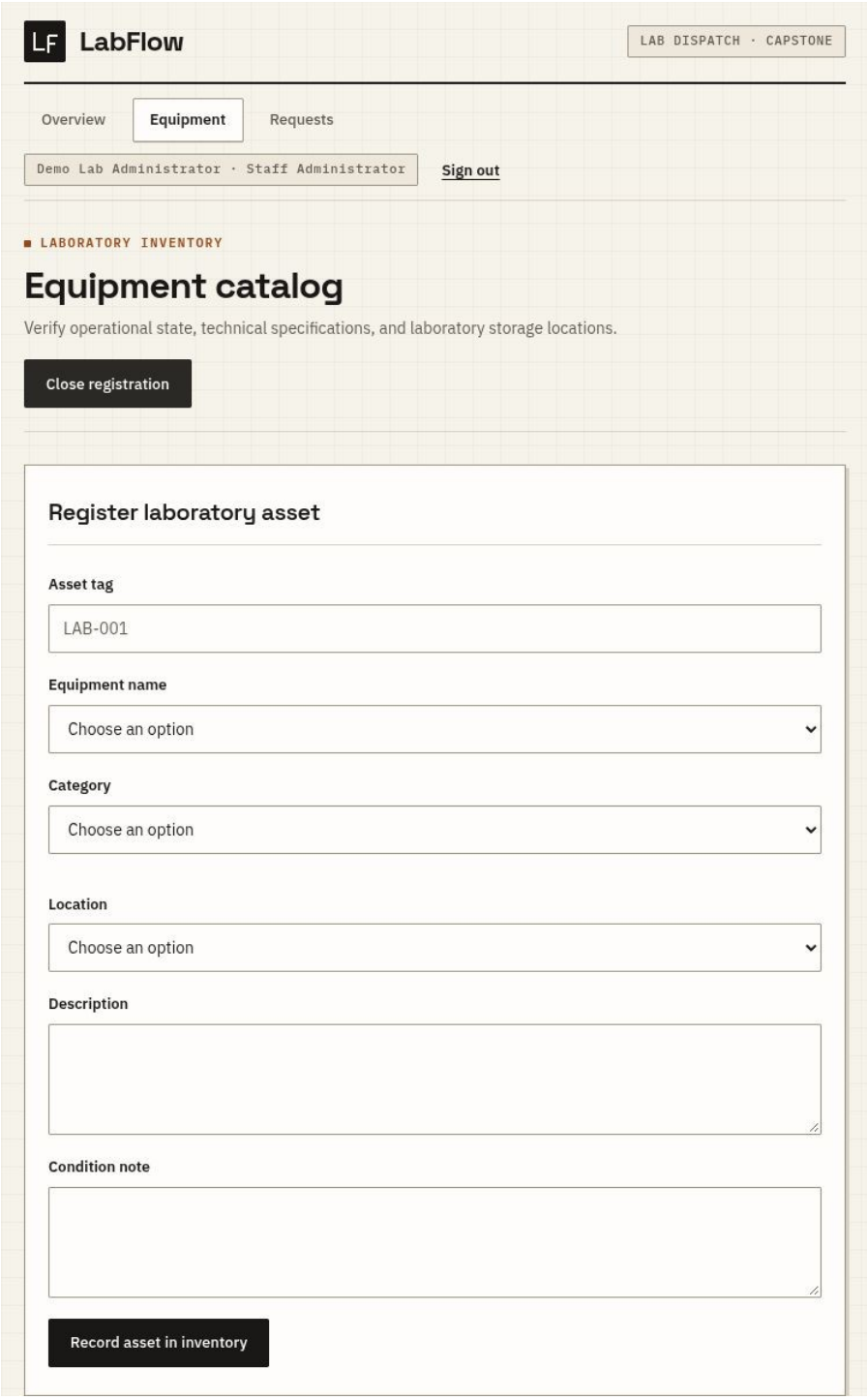


Figure 7. Administrator request management with status, overdue and damage filters. LabFlow application interface, 25 October 2026.

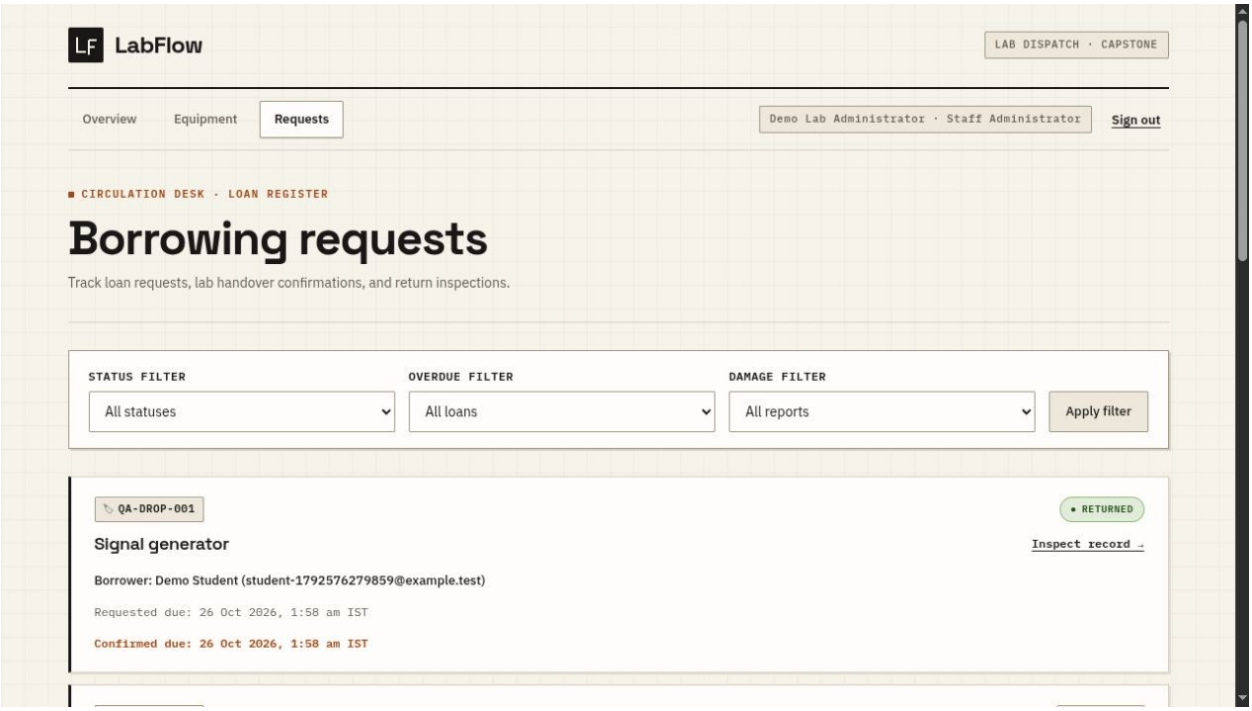


Figure 8. Inspected return, original requested description and retained lifecycle timeline. LabFlow application interface, 25 October 2026.

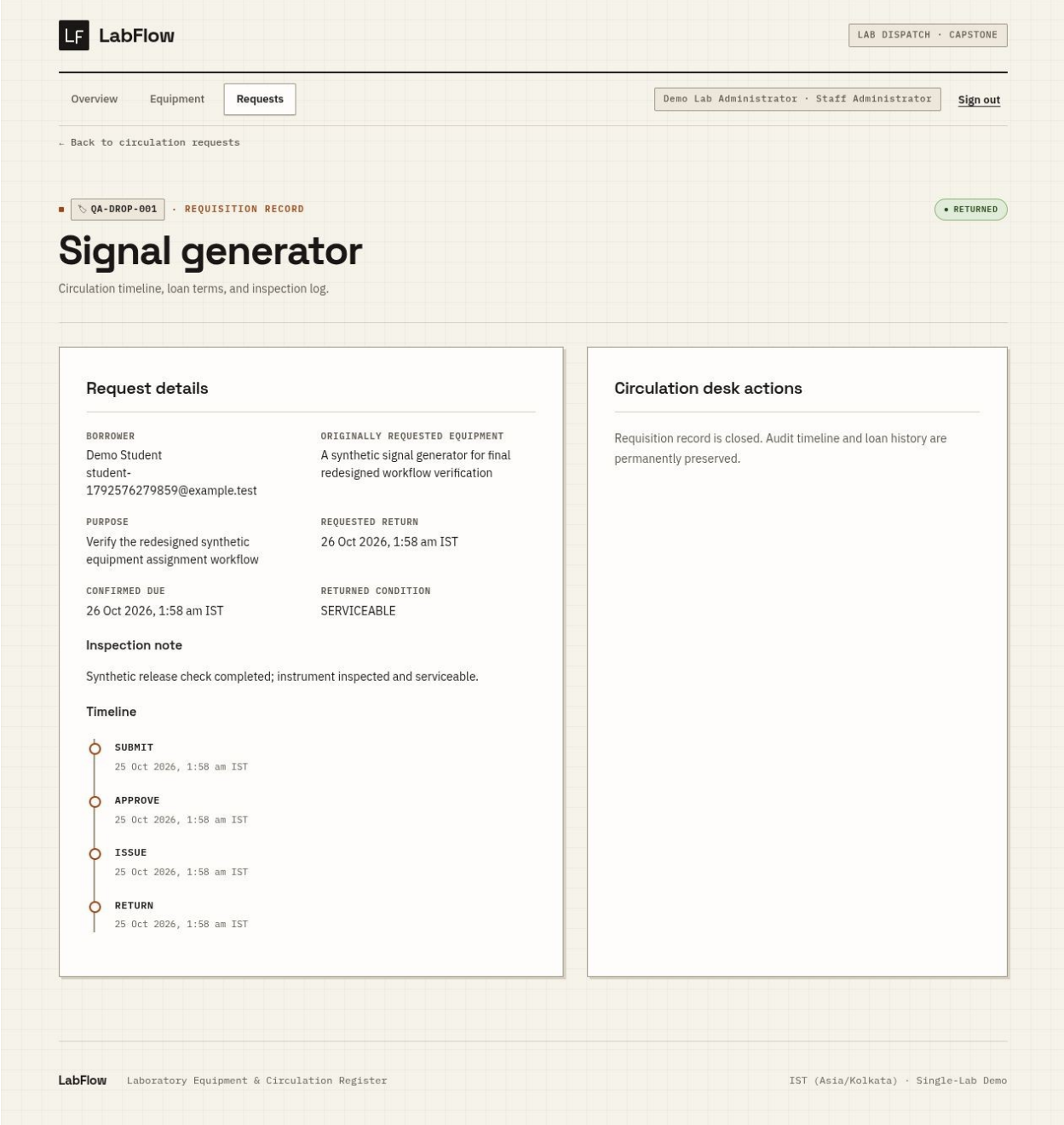


Figure 9. Mobile request form: upper view and scrolled lower fields. LabFlow application interface, 25 October 2026.

